

Murdoku End-to-End: Extracción Visual de Acertijos y Resolución mediante Programación con Restricciones

Camila Adriana Ibarra Cabrera
CC58 – Tópicos en Ciencia
de la Computación

Sofia Gabriel Miranda Cardenas
CC58 – Tópicos en Ciencia
de la Computación

Lizbeth Teresita Olivera Alvarez
CC58 – Tópicos en Ciencia
de la Computación

Resumen—Este trabajo presenta un sistema *end-to-end* que recibe la imagen de un acertijo *Murdoku*, extrae automáticamente su estado inicial y lo resuelve con un modelo formal de Programación con Restricciones (CP). La fase de visión localiza la grilla por contornos, estima su dimensión contando las líneas internas, segmenta las habitaciones agrupando el color de fondo de cada celda con K-Means en CIELAB y las nombra leyendo sus etiquetas con OCR; el mobiliario se clasifica comparando *embeddings* de una red MobileNetV3 preentrenada con plantillas mediante similitud coseno. Los testimonios de las tarjetas se leen con Tesseract y se traducen a restricciones con un analizador determinista en español; un LLM local solo actúa como respaldo. El modelo, implementado en OR-Tools CP-SAT, usa variables booleanas de ocupación canalizadas a coordenadas enteras, restricciones ALLDIFFERENT sobre filas y columnas y restricciones reificadas para la regla del asesino y para cada pista, lo que permite además diagnosticar los errores del jugador en un modo tutor. Sobre un dataset de 10 casos (6×6 y 8×8), la visión obtuvo 98,5 % de precisión y 99,3 % de *recall* en muebles, leyó todos los nombres y pistas, y el sistema identificó al culpable correcto en los 10 casos; cada caso se resuelve en 9–22 ms sin ramificación.

Index Terms—Programación con restricciones, CP-SAT, visión computacional, K-Means, MobileNetV3, OCR, Murdoku, ALLDIFFERENT, restricciones reificadas.

I. INTRODUCCIÓN

Los acertijos lógicos de cuadrícula son un banco de pruebas natural para la Programación con Restricciones (CP): sus reglas se expresan de forma declarativa y su resolución combina propagación y búsqueda [1]. En la práctica, sin embargo, el acertijo llega como una imagen, por lo que un sistema útil debe primero *leer* el tablero y las pistas.

El objetivo de este trabajo es integrar Visión Computacional e Inteligencia Artificial con un solver de CP en un *pipeline* automático que: (i) recibe una imagen (*.png/.jpg*) de un Murdoku, (ii) extrae la grilla, las habitaciones, los muebles y los testimonios, (iii) construye un modelo CP parametrizado, (iv) lo resuelve con OR-Tools CP-SAT [2] y (v) proyecta la solución sobre la imagen original.

Las contribuciones son:

- Un *pipeline* de visión que combina segmentación por contornos, agrupamiento de color no supervisado, OCR

y clasificación por *embeddings* profundos sin entrenar ninguna red.

- Un analizador determinista que traduce testimonios en español a restricciones.
- Un modelo formal de CP para Murdoku, independiente de la instancia, con restricciones globales y reificadas, reutilizado para un tutor interactivo.
- Un dataset de 10 casos con verdad de terreno y una evaluación cuantitativa de la precisión, la robustez y el costo computacional.

II. DESCRIPCIÓN DEL PROBLEMA

Murdoku es un acertijo lógico que combina la estructura del Sudoku con la mecánica de un misterio de asesinato al estilo *Cluedo* [4]. Una instancia consta de una grilla de $N \times N$ celdas particionada en habitaciones (distinguidas por color y rotuladas con etiquetas), muebles impresos en algunas celdas y seis personajes —cinco sospechosos y una víctima—, cada uno con una tarjeta de testimonio. Las reglas son:

1. Cada persona ocupa exactamente una celda.
2. Dos personas no pueden compartir fila ni columna.
3. Nadie puede ubicarse sobre mesas, plantas o la computadora; las sillas sí pueden ocuparse.
4. La víctima está en una habitación con *exactamente un* sospechoso: el asesino.
5. Cada testimonio aporta pistas: columna o fila (ordinales, “última”), habitación, dirección relativa a otra persona (norte, sur, este, oeste), sentarse o no en una silla y estar o no al lado de un mueble (adyacencia ortogonal).

La Fig. 1 muestra el caso “La Tienda” (6×6 , tres habitaciones).

III. ARQUITECTURA DEL SISTEMA

El sistema está organizado en módulos independientes (Fig. 2):

- *src/vision.py*: Fase 1, extracción desde la imagen.
- *src/parser_pistas.py*: traducción de testimonios a restricciones.

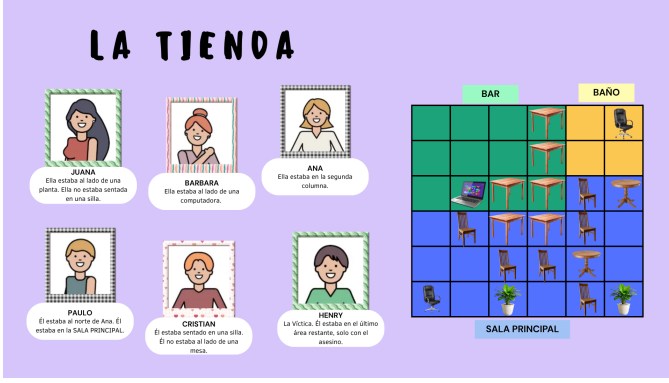


Figura 1. Imagen de entrada del caso “La Tienda”: seis tarjetas en una cuadrícula de 3×2 a la izquierda y el tablero rotulado a la derecha.

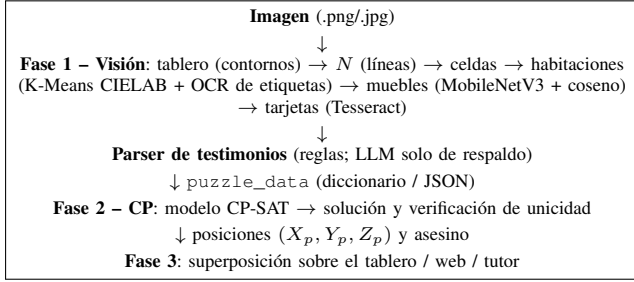


Figura 2. Pipeline end-to-end. Cada fase consume la salida de la anterior sin intervención manual.

- `src/solver.py`: Fase 2, modelo CP-SAT, conteo de soluciones y diagnóstico.
- `src/visualizer.py`: Fase 3, proyección de la solución.
- `src/tutor.py` y `src/agent.py`: tutor interactivo basado en el solver, con redacción opcional por un LLM local.
- `src/catalogo.py` y `casos/casos.json`: catálogo de los 10 casos y su verdad de terreno.
- `main.py` (CLI) y `app.py` (API FastAPI e interfaz web).

El código incluye pruebas automáticas (pytest) del parser, el solver, el catálogo y el tutor, y *scripts* para validar el dataset y evaluar el pipeline.

La estructura intermedia `puzzle_data` es el contrato entre la visión y el solver:

```

1 { "N": 6,
2   "sospechosos": ["Juana", "Barbara", "Ana", "Paulo", "Cristian"],
3   "victima": "Henry",
4   "habitaciones": {(r,c): id_sala, ...},
5   "muebles": {"mesas": [(0,3),...], "sillas": [...],
6               "plantas": [...], "computadora": [(2,1)]},
7   "pistas": [{"tipo": "columna_fija", "persona": "Ana", "columna": 1},
8               {"tipo": "direccion_relativa", "origen": "Paulo",
9               "destino": "Ana", "direccion": "norte"}, ...],
10  "obstaculos": ["mesas", "plantas", "computadora"] }
  
```

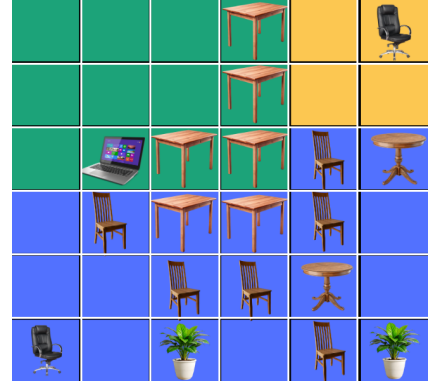


Figura 3. Grilla localizada y segmentada en 6×6 celdas.

IV. FASE 1: VISIÓN COMPUTACIONAL E IA

IV-A. Localización de la grilla

La imagen se convierte a escala de grises y se binariza con umbral inverso ($T = 60$), de modo que el marco negro de la grilla queda en primer plano. Sobre la mitad derecha se extraen los contornos externos con OpenCV [10] y se elige el de mayor área que supere el 20 % de la región y tenga razón de aspecto $w/h \in [0,8; 1,2]$. El rectángulo envolvente se recorta con un margen de 4 px para eliminar el borde.

IV-B. Estimación de N y segmentación de celdas

Sobre el recorte se calcula la máscara de píxeles oscuros (gris < 60) y sus perfiles de proyección vertical y horizontal (fracción de píxeles oscuros por columna y por fila). Los tramos con más del 60 % de píxeles oscuros son líneas; se descartan las pegadas al borde (marco) y N es el número de líneas internas más uno, aceptado solo si coincide en ambos ejes. Con N , el recorte de $H \times W$ se divide en una malla regular de celdas de $\lfloor H/N \rfloor \times \lfloor W/N \rfloor$ píxeles (Fig. 3).

IV-C. Segmentación y rotulado de habitaciones

El color de fondo de cada celda se estima con la *mediana* de los píxeles de sus cuatro esquinas interiores (12 % del lado), lo que evita que el mueble central contamine la estimación. Las muestras se convierten a CIELAB, espacio más cercano a la percepción humana que RGB, y las N^2 medianas se agrupan con K-Means [6] (k = número de habitaciones, 10 inicializaciones).

Para saber *qué* habitación es cada clúster —necesario para pistas como “estaba en la Gerencia”— se leen con Tesseract las etiquetas de color situadas encima y debajo del tablero (–psm 11). Cada etiqueta se asocia a una sala por similitud de texto (difflib, umbral 0,6) y aporta el matiz HSV de su color. Finalmente, las salas se emparejan con los clústeres minimizando la distancia circular de matiz con el algoritmo húngaro [7].

IV-D. Detección y clasificación de mobiliario

Detección de presencia. Se descartan las celdas de suelo liso: si la desviación estándar de la intensidad en la región central (64 % del área) es $\sigma < 14$, la celda se considera vacía.

Clasificación por *embeddings*. De cada celda con objeto se extrae un vector de características con MobileNetV3-Small [5] preentrenada en ImageNet, reemplazando el clasificador por la identidad. El recorte se redimensiona a 128×128 , se normaliza con las estadísticas de ImageNet y produce un *embedding* $e \in \mathbb{R}^{576}$ normalizado en L_2 . Lo mismo se hace una sola vez con 6 plantillas (dos sillas, dos mesas, planta y computadora). La clase asignada es

$$\hat{k} = \arg \max_{t \in \mathcal{T}} \cos(e_{\text{celda}}, e_t) \quad \text{si} \quad \max_t \cos(\cdot) \geq 0,45. \quad (1)$$

Este enfoque *one-shot* por similitud no requiere entrenamiento: un mueble nuevo se agrega con una sola imagen de plantilla.

IV-E. Lectura de tarjetas y traducción de testimonios

OCR. Las seis tarjetas ocupan cajas fijas, definidas en proporción a una imagen de referencia de 1920×1080 y escaladas al tamaño real. Cada caja se pasa a gris, se amplía $\times 2$ con interpolación cúbica, se binariza con Otsu [8] y se lee con Tesseract [9] (modelo *spa*, *-psm 6*). El testimonio empieza en la primera línea que arranca con “Él/Ella/La”; el nombre es la palabra en mayúsculas de la línea anterior. La víctima es la tarjeta que contiene “la víctima”.

Parser determinista. Los testimonios usan un vocabulario cerrado, por lo que se traducen con reglas sobre texto normalizado (minúsculas, sin tildes): expresiones regulares para direcciones (“al norte de Ana”), ordinales de fila o columna (“segunda”, “última”), adyacencia (“al lado de una planta”), sillas y nombres de salas, con detección de negación (“no estaba”). Los nombres de personas se resuelven con coincidencia difusa (*diffli*, umbral 0,75) para tolerar errores de OCR.

Respaldo con LLM. Solo las oraciones que ninguna regla reconoce se envían a un LLM local (Llama 3.2 3B vía Ollama, temperatura 0, salida JSON). Su respuesta se valida estrictamente (personas y salas existentes, índices dentro del tablero) antes de usarse. En los 10 casos del dataset el respaldo no fue necesario.

IV-F. Dataset

Se construyó un dataset de 10 casos originales (Tabla I): 8 de 6×6 y 2 de 8×8 , con tres habitaciones cada uno y de dificultad creciente. Su verdad de terreno (mapa de salas, muebles, testimonios y culpable) está en *casos/casos.json*; un *script* verifica con el solver que cada caso tenga solución única.

IV-G. Métricas de la visión

Para cada caso se mide:

- ***N***: la dimensión estimada coincide con la real.
- **Salas**: porcentaje de celdas con la habitación correcta.
- **Muebles**: una detección es acierto (TP) si coinciden tipo y celda; precisión = TP/detectados y recall = TP/reales.
- **Nombres**: nombres de las 6 tarjetas leídos correctamente.
- **Pistas**: fracción de las pistas esperadas que se extrajeron.

La Fig. 4 resume la evaluación completa del *pipeline* (*scripts/evaluar.py*). *N* se estimó bien en los 10 casos, se leyeron los 60 nombres y el 100 % de las pistas

Tabla I
DATASET DE CASOS

Caso	Dificultad	<i>N</i>	Muebles
01 La Tienda	Intermedio	6	19
02 La Oficina	Principiante	6	11
03 La Biblioteca Colonial	Principiante	6	10
04 El Centro Tecnológico	Intermedio	6	12
05 La Galería Minimalista	Intermedio	6	11
06 El Cafetín Universitario	Intermedio	6	14
07 La Agencia de Diseño	Avanzado	6	12
08 El Coworking Urbano	Avanzado	6	12
09 La Mansión de Cristal	Experto	8	17
10 La Sala de Prensa	Experto	8	16

Calidad de la extracción por caso					
	Salas (% celdas)	Muebles precisión	Muebles recall	Nombres OCR	Pistas recall
01 La Tienda (6×6)	100%	100%	100%	100%	100%
02 La Oficina (6×6)	100%	100%	100%	100%	100%
03 La Biblioteca Colonial (6×6)	100%	100%	100%	100%	100%
04 El Centro Tecnológico (6×6)	33%	100%	100%	100%	100%
05 La Galería Minimalista (6×6)	100%	100%	100%	100%	100%
06 El Cafetín Universitario (6×6)	100%	100%	100%	100%	100%
07 La Agencia de Diseño (6×6)	100%	100%	100%	100%	100%
08 El Coworking Urbano (6×6)	100%	100%	100%	100%	100%
09 La Mansión de Cristal (8×8)	100%	100%	100%	100%	100%
10 La Sala de Prensa (8×8)	100%	88%	94%	100%	100%
Promedio	93%	99%	99%	100%	100%

Más oscuro = mejor; las celdas claras señalan los casos a corregir
Corrida «base» - commit cc816e4 - 10 casos

Figura 4. Calidad de la extracción por caso sobre las imágenes digitales.

esperadas, y en muebles se obtuvieron 133 aciertos sobre 135 detecciones y 134 muebles reales: precisión 98,5 % y recall 99,3 %.

Análisis de errores.

- **Caso 04, salas (33 %).** K-Means agrupa las celdas correctamente (100 % tomando la mejor correspondencia de etiquetas), pero el rotulado por color asignó nombres permutados a los clústeres. Con salas intercambiadas el modelo admite más de una solución, aunque el culpable obtenido sigue siendo el correcto.
- **Caso 10, muebles.** Las tres discrepancias están en la última columna (Fig. 5). La inspección muestra que la imagen dibuja una planta en (3,7) y una silla en (5,7), con (6,7) vacía: la visión reporta lo que está dibujado y el error está en la imagen respecto al catálogo, no en el clasificador.

IV-H. Robustez ante distintas condiciones de imagen

Las imágenes del dataset son digitales. Para medir la robustez se generaron 10 variantes de cada uno de los 10 casos que simulan condiciones de captura distintas (Fig. 6), y se evaluaron las etapas de tablero, *N*, salas (K-Means) y muebles: 110 imágenes en total. La Tabla II muestra los resultados.

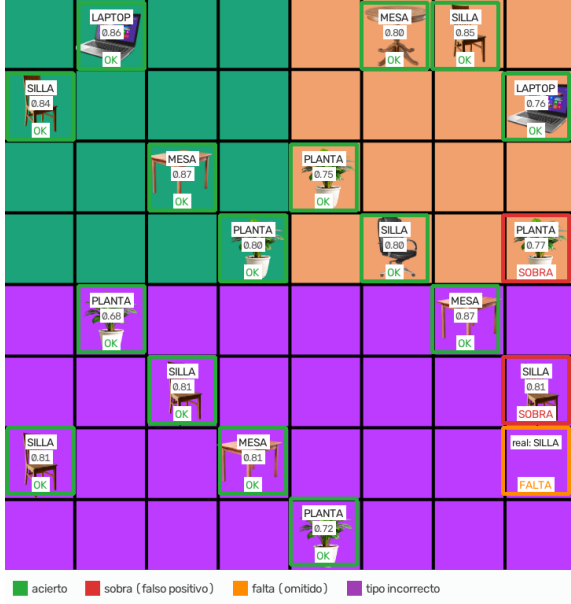


Figura 5. Detección por celda en el caso 10 con la similitud coseno de la plantilla ganadora. En rojo y naranja, las discrepancias con el catálogo.

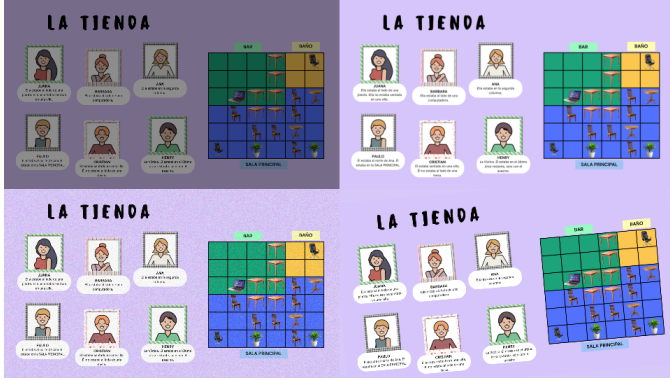


Figura 6. Ejemplos de variantes: baja iluminación, desenfoque, ruido gaussiano y rotación de 5°.

El sistema es robusto a cambios de brillo y contraste, ruido, compresión, desenfoque y cambio de escala. Las fallas son informativas:

- **Baja iluminación:** los muebles perdidos no son errores de clasificación sino de *detección*: con menos contraste, la desviación estándar del centro de la celda cae bajo el umbral fijo $\sigma = 14$. Un umbral relativo a la imagen o una ecualización CLAHE previa lo corregiría.
- **Rotación y perspectiva:** las líneas de la grilla dejan de ser horizontales y el conteo de líneas por proyección falla (por eso N se toma del catálogo en esos casos). Además, el recorte por rectángulo alineado a los ejes desfasa la malla, lo que produce errores de sala en los bordes y, a 5°, fragmentos de muebles vecinos detectados como falsos positivos. Se requiere una corrección de perspectiva con los cuatro vértices del contorno (approxPolyDP + warpPerspective).

Tabla II
ROBUSTEZ DE LA VISIÓN (10 CASOS POR CONDICIÓN)

Condición	N	Salas	Prec.	Recall	Perf.
Original	10/10	1,000	0,985	0,993	9
Oscura ($\alpha = 0,55$)	10/10	1,000	0,983	0,843	1
Clara (+50)	10/10	1,000	0,985	0,993	9
Contraste ($\alpha = 1,4$)	10/10	1,000	0,985	0,993	9
Desenfoque ($k = 7$)	10/10	1,000	0,963	0,970	8
Ruido ($\sigma = 20$)	10/10	1,000	0,985	0,955	6
JPEG (calidad 25)	10/10	1,000	0,985	0,993	9
Escala 0,5	10/10	1,000	0,985	0,993	9
Perspectiva leve	0/10	0,997	0,985	0,993	8
Rotación 2°	0/10	0,869	0,985	0,993	1
Rotación 5°	0/10	0,797	0,715	0,918	0

Salas: mejor correspondencia de clústeres. Perf.: casos sin ningún error de sala ni de mueble.

V. FASE 2: MODELO DE PROGRAMACIÓN CON RESTRICCIONES

V-A. Parámetros de la instancia

Todos los parámetros provienen de `puzzle_data`; el modelo no contiene datos de ningún caso concreto:

- N : dimensión de la grilla; $\mathcal{C} = \{0, \dots, N-1\}^2$ celdas.
- $\mathcal{S}$: sospechosos; v : víctima; $\mathcal{P} = \mathcal{S} \cup \{v\}$.
- $\mathcal{H}$: habitaciones; $\rho: \mathcal{C} \rightarrow \mathcal{H}$ mapa de habitaciones.
- $M_k \subseteq \mathcal{C}$: celdas con mueble de tipo k .
- $\mathcal{O} = \bigcup_{k \in K_{obs}} M_k$: celdas obstáculo (mesas, plantas, computadora).
- $\mathcal{A}(M) = \{(r', c') \in \mathcal{C} : |r - r'| + |c - c'| = 1, (r, c) \in M\}$: vecindad ortogonal de un conjunto de celdas.
- Π : conjunto de pistas extraídas.

V-B. Variables y dominios

$$b_{p,r,c} \in \{0, 1\} \quad \forall p \in \mathcal{P}, (r, c) \in \mathcal{C} \quad (2)$$

$$X_p \in \{0, \dots, N-1\} \quad (\text{fila de } p) \quad (3)$$

$$Y_p \in \{0, \dots, N-1\} \quad (\text{columna de } p) \quad (4)$$

$$Z_p \in \{\min \mathcal{H}, \dots, \max \mathcal{H}\} \quad (\text{habitación de } p) \quad (5)$$

$$\ell_k \in \{0, 1\} \quad \forall k \in \Pi \text{ (literal de pista)} \quad (6)$$

$b_{p,r,c} = 1$ si y solo si p ocupa la celda (r, c) . Además se usan booleanas auxiliares ν_h y $\sigma_{s,h}$ para la regla del asesino (§V-D).

V-C. Restricciones

(C1) Ocupación única — cada persona en exactamente una celda:

$$\sum_{(r,c) \in \mathcal{C}} b_{p,r,c} = 1 \quad \forall p \in \mathcal{P}. \quad (7)$$

(C2-C3) Unicidad de filas y columnas:

$$\sum_{p \in \mathcal{P}} \sum_c b_{p,r,c} \leq 1 \quad \forall r, \quad (8)$$

$$\sum_{p \in \mathcal{P}} \sum_r b_{p,r,c} \leq 1 \quad \forall c. \quad (9)$$

Tabla III
TRADUCCIÓN DE PISTAS A RESTRICCIONES R_k

Tipo de pista	Restricción
columna_fija(p, j)	$Y_p = j$
fila_fija(p, i)	$X_p = i$
habitacion_fija(p, h)	$Z_p = h$
direccion($p, q, norte$)	$X_p < X_q$
direccion(p, q, sur)	$X_p > X_q$
direccion($p, q, este$)	$Y_p > Y_q$
direccion($p, q, oeste$)	$Y_p < Y_q$
sobre_objeto(p, k)	$\sum_{(r,c) \in M_k} b_{p,r,c} = 1$
no_sobre_objeto(p, k)	$\sum_{(r,c) \in M_k} b_{p,r,c} = 0$
adyacente_a(p, k)	$\sum_{(r,c) \in \mathcal{A}(M_k)} b_{p,r,c} \geq 1$
no_adyacente_a(p, k)	$\sum_{(r,c) \in \mathcal{A}(M_k)} b_{p,r,c} = 0$

Junto con (C1) y la canalización (C4), equivalen a las restricciones globales

$$\text{ALLDIFFERENT}(X_p : p \in \mathcal{P}), \quad \text{ALLDIFFERENT}(Y_p : p \in \mathcal{P}) \quad (10)$$

Se expresan en su descomposición booleana *at-most-one*, que CP-SAT reconoce y propaga como restricción de cardinalidad mediante cláusulas y detección de cliques [3]. Esta codificación es eficiente porque la misma matriz b se reutiliza en las pistas sobre muebles, que se reducen a sumas lineales sin restricciones ELEMENT adicionales.

(C4) **Canalización** (*channeling*) entre la representación booleana y la entera:

$$X_p = \sum_{(r,c)} r b_{p,r,c}, \quad Y_p = \sum_{(r,c)} c b_{p,r,c}, \quad (11)$$

$$Z_p = \sum_{(r,c)} \rho(r, c) b_{p,r,c} \quad \forall p \in \mathcal{P}. \quad (12)$$

Dado (C1), (C4) implementa la restricción global $Z_p = \text{ELEMENT}(\rho, \text{celda}(p))$ como una suma lineal.

(C5) **Obstáculos**:

$$b_{p,r,c} = 0 \quad \forall p \in \mathcal{P}, (r, c) \in \mathcal{O}. \quad (13)$$

(C6) **Regla del asesino** (reificada): si la víctima está en h , exactamente un sospechoso está en h :

$$\nu_h \Leftrightarrow (Z_v = h) \quad \forall h \in \mathcal{H}, \quad (14)$$

$$\sigma_{s,h} \Leftrightarrow (Z_s = h) \quad \forall s \in \mathcal{S}, h \in \mathcal{H}, \quad (15)$$

$$\nu_h \Rightarrow \sum_{s \in \mathcal{S}} \sigma_{s,h} = 1 \quad \forall h \in \mathcal{H}. \quad (16)$$

El asesino es el único s con $Z_s = Z_v$.

(C7) **Pistas** (semirreificadas): cada pista $k \in \Pi$ se traduce a una restricción R_k según la Tabla III y se condiciona a su literal:

$$\ell_k \Rightarrow R_k \quad \forall k \in \Pi. \quad (17)$$

Al resolver un caso se fija $\ell_k = 1$ para toda pista. Un tipo de pista, persona, sala o dirección desconocida produce un error explícito en lugar de ignorarse, porque omitir una pista en silencio podría devolver una solución incorrecta.

V-D. Uso de restricciones reificadas

La reificación se usa en dos lugares, cada uno por una razón concreta:

Regla del asesino (C6). Es *condicional* a una decisión del propio modelo —la habitación de la víctima—, por lo que no puede expresarse como restricción dura fija. Se implementa con literales de activación (*OnlyEnforceIf*) de CP-SAT en ambos sentidos.

Pistas (C7). Para resolver, todas las pistas son hechos ciertos y bastaría imponerlas como restricciones duras; fijar $\ell_k = 1$ hace que la presolución las elimine, sin costo de propagación. La reificación aporta valor en el **modo tutor**: las ubicaciones del jugador se agregan como literales $m_p = b_{p,r,c}$ y, junto con los ℓ_k , se pasan al solver como *supuestos* (*AddAssumptions*). Si el modelo es infactible, CP-SAT devuelve un subconjunto de supuestos suficiente para la contradicción (*SufficientAssumptionsForInfeasibility*), es decir, qué ubicaciones y qué testimonios chocan, lo que se muestra al jugador sin revelar la solución. Las pistas se reifican solo en un sentido ($\ell_k \Rightarrow R_k$), porque no se necesita deducir que una pista es falsa.

El tutor también usa el modelo para calcular las celdas factibles de la persona más acotada (una resolución con un supuesto por celda candidata) y ofrecer ayuda progresiva en tres niveles.

V-E. Implementación en OR-Tools CP-SAT

El modelo se construye en la función `_construir_modelo()` de `src/solver.py`. Cada restricción formal corresponde a una o pocas líneas de la API de CP-SAT; los listados siguientes reproducen el código con nombres abreviados. El Listado 1 muestra las variables y las restricciones estructurales (C1–C5). Todos los rangos y conjuntos (N , personas, mapa de salas, muebles) se leen de `puzzle_data`, por lo que el mismo código sirve para cualquier instancia.

Listado 1. Variables y restricciones C1–C5

```

1 model = cp_model.CpModel()
2 celdas = [(r, c) for r in range(N) for c in range(N)]
3
4 # Booleanas de ocupacion b[p,r,c]
5 celda = {(p, r, c): model.NewBoolVar(f"celda_{p}_{r}_{c}")
6          for p in range(P) for (r, c) in celdas}
7
8 # C1: cada persona en exactamente una celda
9 for p in range(P):
10     model.Add(sum(celda[p, r, c] for r, c in celdas) ==
11               1)
12
13 # C2-C3: AllDifferent en filas y columnas (at-most-one)
14 for r in range(N):
15     model.Add(sum(celda[p, r, c] for p in range(P)
16                   for c in range(N)) <= 1)
17 for c in range(N):
18     model.Add(sum(celda[p, r, c] for p in range(P)
19                   for r in range(N)) <= 1)
20
21 # C4: canalizacion a fila X, columna Y y habitacion Z
22 X = [model.NewIntVar(0, N - 1, f"X_{p}") for p in
23      personas]
24 Y = [model.NewIntVar(0, N - 1, f"Y_{p}") for p in
25      personas]
```

```

23 Z = [model.NewIntVar(min(ids_hab), max(ids_hab), f"Z_{p}")
24 )
25 for p in personas:
26     model.Add(X[p] == sum(r * celda[p, r, c] for r, c in
27         celdas))
28     model.Add(Y[p] == sum(c * celda[p, r, c] for r, c in
29         celdas))
30     model.Add(Z[p] == sum(mapa_hab[r, c] * celda[p, r, c]
31         for r, c in celdas))
32 # C5: obstaculos (mesas, plantas, computadora)
33 for tipo in obstaculos:
34     for r, c in muebles.get(tipo, []):
35         for p in range(P):
36             model.Add(celda[p, r, c] == 0)

```

El Listado 2 muestra las restricciones reificadas. En CP-SAT la reificación se expresa con `OnlyEnforceIf(lit)`: la restricción solo se exige cuando el literal `lit` es verdadero. Para la regla del asesino (C6) se reifica la igualdad $Z_v = h$ en ambos sentidos; para las pistas (C7), cada una recibe su literal ℓ_k , que se fija a 1 al resolver o se pasa como supuesto en el modo tutor.

Listado 2. Restricciones reificadas C6–C7 y diagnóstico

```

1 # C6: victima en h => exactamente un sospechoso en h
2 for h in ids_hab:
3     vic_en_h = model.NewBoolVar(f"vic_en_hab_{h}")
4     model.Add(Z[v] == h).OnlyEnforceIf(vic_en_h)
5     model.Add(Z[v] != h).OnlyEnforceIf(vic_en_h.Not())
6     s_en_h = []
7     for s in range(S):
8         b = model.NewBoolVar(f"s_{s}_en_hab_{h}")
9         model.Add(Z[s] == h).OnlyEnforceIf(b)
10        model.Add(Z[s] != h).OnlyEnforceIf(b.Not())
11        s_en_h.append(b)
12    model.Add(sum(s_en_h) == 1).OnlyEnforceIf(vic_en_h)
13
14 # C7: cada pista k condicionada a su literal l_k
15 for pista in pistas:
16     lit = model.NewBoolVar(f"pista_{k}")
17     if pista["tipo"] == "columna_fija":
18         model.Add(Y[p] == pista["columna"]).OnlyEnforceIf(
19             lit)
20     elif pista["tipo"] == "direccion_relativa": # norte
21         model.Add(X[o] < X[d]).OnlyEnforceIf(lit)
22     elif pista["tipo"] == "adyacente_a":
23         vec = vecinos_de(muebles[pista["objeto"]], N)
24         model.Add(sum(celda[p, r, c] for r, c in vec) >=
25             1
26             ).OnlyEnforceIf(lit)
27     # ... resto de tipos de la Tabla de pistas
28
29 # Resolver: todas las pistas activas
30 for lit in literales:
31     model.Add(lit == 1)
32
33 # Tutor: pistas y jugadas como supuestos
34 model.AddAssumptions(literales + jugadas)
35 if solver.Solve(model) == cp_model.INFEASIBLE:
36     nucleo = solver.SufficientAssumptionsForInfeasibility()

```

Además de `resolver_murdoku()`, el módulo ofrece `contar_soluciones()`, que enumera soluciones con `enumerate_all_solutions` y un `callback` para verificar la unicidad de cada caso, y `celdas_factibles()`, que el tutor usa para calcular dónde puede estar cada persona.

V-F. Solución del caso “La Tienda”

La Tabla IV muestra las posiciones obtenidas: Barbara es la única sospechosa en el Bar junto a Henry, por lo que es la asesina.

Tabla IV
SOLUCIÓN DEL CASO “LA TIENDA” (ÍNDICES DESDE 0)

Persona	Fila	Col.	Habitación
Juana	5	3	Sala Principal
Barbara	2	0	Bar (asesina)
Ana	4	1	Sala Principal
Paulo	3	4	Sala Principal
Cristian	0	5	Baño
Henry	1	2	Bar (víctima)

VI. ANÁLISIS DE COMPLEJIDAD Y TIEMPOS

VI-A. Tamaño del espacio de búsqueda

Sin restricciones, colocar $|\mathcal{P}|$ personas en N^2 celdas da $(N^2)^{|\mathcal{P}|}$ configuraciones: para $N = |\mathcal{P}| = 6$ son $36^6 \approx 2,18 \times 10^9$. Las restricciones (C1–C3) lo reducen a las asignaciones inyectivas de filas y columnas, $\left(\frac{N!}{(N-|\mathcal{P}|)!}\right)^2$: para $N = 6$ son $(6!)^2 = 518400$ y para $N = 8$, $(8!/2!)^2 \approx 4,06 \times 10^8$. Con obstáculos y la regla del asesino, pero sin pistas, los casos de 6×6 aún tienen entre 7680 y 123840 soluciones (Tabla V); las pistas las reducen a una.

El problema general de satisfacción de restricciones es NP-completo [1], por lo que no se espera un algoritmo polinomial para la familia generalizada. Sin embargo, los Murdoku están diseñados para resolverse por deducción, lo que favorece a la propagación.

VI-B. Tamaño del modelo

El número de variables es

$$|V| = \underbrace{|\mathcal{P}|N^2}_b + \underbrace{3|\mathcal{P}|}_{X,Y,Z} + \underbrace{|\mathcal{H}|(1+|\mathcal{S}|)}_{\nu,\sigma} + \underbrace{|\Pi|}_{\ell} = O(|\mathcal{P}|N^2), \quad (18)$$

que para “La Tienda” da $216 + 18 + 18 + 8 = 260$ variables, como reporta el modelo generado. El número de restricciones es $O(N + |\mathcal{P}||\mathcal{H}| + |\Pi|)$, pero (C2–C4) involucran $O(|\mathcal{P}|N)$ y $O(N^2)$ literales cada una, de modo que el tamaño total del modelo es $O(|\mathcal{P}|N^2)$.

VI-C. Tiempos de respuesta

Las mediciones se hicieron en un Intel Core i5-6500 (3,2 GHz, 4 núcleos) con 20 GB de RAM, Windows 10, Python 3.13 y OR-Tools 9.15 con parámetros por defecto.

La Tabla V resume el solver sobre los 10 casos (30 ejecuciones cada uno). Todos tienen solución única, el culpable coincide con el del catálogo y **ninguno requiere ramificación**: la presolución y la propagación fijan todas las variables. El tiempo reportado por CP-SAT está entre 9 y 22 ms, y el tiempo total, incluida la construcción del modelo en Python, entre 17 y 35 ms.

Ablación de pistas. La columna “Nec.” indica cuántas pistas son imprescindibles: al eliminar cualquiera de ellas aparece más de una solución. En los casos de 6×6 solo entre 3 y 7 pistas lo son; el resto son redundantes dadas las demás, lo que es propio de un acertijo diseñado con varias rutas de

Tabla V
SOLVER SOBRE LOS 10 CASOS DEL DATASET

Caso	V	Restr.	\Pi	Nec.	Sol. sin \Pi	t (ms)
01	260	157	8	3	7 680	13,7
02	264	141	12	4	43 680	22,3
03	264	141	12	5	44 400	14,6
04	262	143	10	7	123 840	16,0
05	261	135	9	5	11 520	13,2
06	264	153	12	4	92 160	12,9
07	264	147	12	5	36 480	11,2
08	261	135	9	5	20 160	9,1
09	434	173	14	10	>167 487	20,1
10	435	169	15	10	>194 276	18,2

Nec.: pistas cuya eliminación rompe la unicidad. Sol. sin \Pi: soluciones sin pistas (>: enumeración cortada a 60 s).
t: tiempo medio de CP-SAT. Todos: 0 ramas y 0 conflictos.

Tabla VI
ESCALABILIDAD DEL MODELO ($|\mathcal{P}| = N$)

N	Variables	Restricciones	Tiempo (ms)
4	89	53	17,5 ± 2,5
6	253	77	34,3 ± 3,2
8	561	101	71,9 ± 7,1
10	1 061	125	146,4 ± 11,1
12	1 801	149	302,3 ± 17,8
15	3 466	185	779,2 ± 11,4
20	8 121	245	3 021 ± 39
25	15 776	305	5 908 ± 152
30	27 181	365	10 615 ± 588

deducción. En los casos “Experto” de 8×8 , 10 de las 14–15 pistas son necesarias.

Escalabilidad. Para medir el crecimiento se generaron instancias sintéticas de $N \times N$ con $|\mathcal{P}| = N$ personas, 3 habitaciones y una pista (Tabla VI y Fig. 7, 10 ejecuciones). El crecimiento es polinomial en la práctica, dominado por el tamaño $O(N^3)$ del modelo cuando $|\mathcal{P}| = N$: incluso con 30×30 y 27 181 variables se resuelve en unos 10 s.

Costo del pipeline completo. La Fig. 8 muestra los tiempos de la evaluación completa por caso: la visión con OCR toma entre 2,2 y 3,0 s y el solver entre 9 y 26 ms. De la visión, la localización y K-Means toman unos 20 ms y la extracción de *embeddings* entre 120 y 350 ms en CPU; el resto corresponde a Tesseract. El solver representa menos del 1 % del tiempo total.

VII. FASE 3: INTEGRACIÓN Y VISUALIZACIÓN

La integración es automática: `extraer_puzzle_completo()` devuelve `puzzle_data`, que `resolver_murdoku()` consume directamente, y su resultado alimenta a `dibujar_solucion_sobre_tablero()`. El usuario solo elige la imagen.

La solución se proyecta sobre el recorte original del tablero (Fig. 9): cada persona se marca con un rectángulo semitransparente (opacidad 45 %, para que el color de la habitación siga visible) y su nombre; el asesino se resalta en rojo con borde grueso y la víctima en gris oscuro.

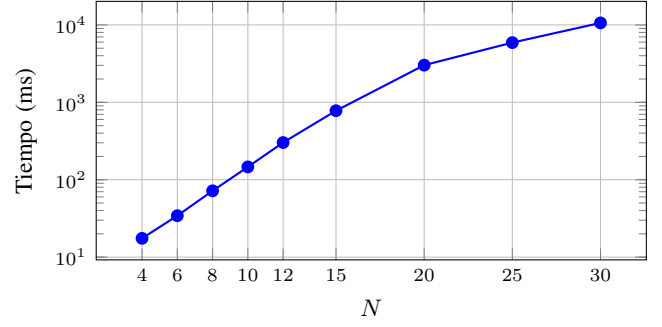


Figura 7. Tiempo total de construcción y resolución en función de N (escala logarítmica).

Tiempo de ejecución por caso



Figura 8. Tiempo de visión + OCR y de CP-SAT por caso.

El sistema se ofrece en dos modos:

- **CLI** (`python main.py -caso caso_02`): ejecuta el *pipeline*, imprime lo extraído, indica si la solución es única y guarda la imagen resuelta.
- **Web** (`python app.py`): API REST con FastAPI (`/api/casos`, `/api/analizar`, `/api/resolver`, `/api/tutor`, `/api/validar`) y una interfaz con galería de casos y tablero interactivo. El jugador ubica personas, el solver valida su avance (§V-D) y ofrece pistas progresivas; opcionalmente, un LLM local reformula el mensaje, y una guardia descarta su texto si agrega números, nombres o razonamiento nuevos.

VIII. DISCUSIÓN Y LIMITACIONES

- **Corrección de perspectiva.** La localización usa el rectángulo envolvente alineado a los ejes y el conteo de líneas por proyección, suficientes para imágenes digitales y fotos frontales, pero no para rotaciones o perspectiva (Tabla II). La mejora directa es una transformación de cuatro puntos sobre el contorno de la grilla.
- **Umbral de detección fijo.** El umbral $\sigma = 14$ falla con poca luz y debe hacerse adaptativo.



Figura 9. Solución del caso 09 (8×8) proyectada sobre la imagen original. En rojo, la asesina (Irene); en gris, la víctima.

- **Información del catálogo.** La CLI y la evaluación pasan N y la lista de nombres de salas del catálogo. N se estimó correctamente en las 10 imágenes digitales, por lo que podría tomarse siempre de la imagen; los nombres de sala sirven como vocabulario para rotular clústeres e interpretar pistas, y sin ellos las salas se numeran por color.
- **Diseño fijo de tarjetas.** Las cajas de las tarjetas son proporciones fijas del diseño del dataset; otro diseño requeriría detectarlas por contornos.
- **Dataset.** Las variantes de robustez son sintéticas; deben complementarse con fotografías de versiones impresas.

IX. CONCLUSIONES

Se desarrolló un sistema *end-to-end* que lee la imagen de un Murdoku y lo resuelve sin intervención manual. Sin entrenar ninguna red, la combinación de K-Means en CIELAB, *embeddings* de MobileNetV3, Tesseract y un parser determinista extrajo correctamente los nombres y las pistas de los 10 casos, con 98,5 % de precisión y 99,3 % de *recall* en muebles, y el sistema identificó al culpable correcto en todos. El modelo CP, con representación booleana canalizada a coordenadas enteras, restricciones ALLDIFFERENT y reificación donde aporta —la regla del asesino y el diagnóstico de errores del jugador—, resolvió cada caso en 9–22 ms solo por propagación y escala a grillas de 30×30 en segundos. El cuello de botella no es el solver sino la percepción: el trabajo futuro debe centrarse en la corrección de perspectiva, umbrales adaptativos y un dataset de fotografías reales.

REFERENCIAS

- [1] F. Rossi, P. van Beek y T. Walsh, Eds., *Handbook of Constraint Programming*. Amsterdam: Elsevier, 2006.
- [2] L. Perron y V. Furnon, “OR-Tools,” Google, 2024. [En línea]. Disponible: <https://developers.google.com/optimization>

- [3] L. Perron, F. Didier y S. Gay, “The CP-SAT-LP solver,” en *Proc. 29th Int. Conf. Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs, vol. 280, 2023, pp. 3:1–3:2.
- [4] “Murdoku,” *Wikipedia*. [En línea]. Disponible: <https://nl.wikipedia.org/wiki/Murdoku>
- [5] A. Howard *et al.*, “Searching for MobileNetV3,” en *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [6] S. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [7] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [8] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
- [9] R. Smith, “An overview of the Tesseract OCR engine,” en *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [10] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.